

Frappe Operator on OpenShift: Full Deployment Manual

This guide provides the authoritative, step-by-step workflow for deploying the Frappe Operator and ERPNext in an enterprise OpenShift 4.x environment with external Redis and MariaDB.

1. Prerequisites

- Access to an OpenShift 4.x cluster (v4.10+ recommended).
 - `oc` CLI authenticated (`oc login`).
 - `helm` CLI (v3+) installed.
 - Cluster-admin permissions or equivalent RBAC to create Namespaces, CRDs, and cluster-wide RBAC.
 - External Redis and MariaDB instances are already running and accessible from the cluster.
-

2. Prepare the Infrastructure

All operator components should reside in a dedicated namespace to maintain clean security boundaries.

```
oc new-project frappe-operator-system
```

3. Install Frappe Operator

Install the Frappe Operator using the official Helm chart.

```
# Add the Frappe Operator repository
helm repo add frappe-operator https://vyogotech.github.io/frappe-operator/helm-repo
helm repo update

# Install the operator (disabling nested dependencies as they were externally managed)
helm upgrade --install frappe-operator frappe-operator/frappe-operator \
  --namespace frappe-operator-system \
  --set mariadb-operator.enabled=false \
  --set keda.enabled=false \
  --wait
```

4. Security Note: SCC Compatibility

The Frappe Operator is natively compatible with OpenShift's `restricted-v2` Security Context Constraint (SCC).

- **Dynamic UIDs:** The operator uses `nil` defaults for `runAsUser`, allowing OpenShift to automatically assign a compliant UID.
- **Filesystem Access:** It uses platform-aware volume permissions for GID 0 compatibility.

> [!TIP]

> For deep technical details on SCC handling, see the [OpenShift Technical Guide](#).

5. Preparing OpenShift-Compatible Container Images

OpenShift requires containers to run as non-root (GID 0). The official images must be patched to work with OpenShift. We have the patched and tested base image in our container registry. We will be using this base image to create a new image with your custom apps. The full containerfile for creating this base image can be found [here](#).

5.1 Custom-Apps Image – Two-Stage Build

To include custom applications, use a two-stage build to keep the runtime image lean.

1. Define apps.json

```
{
  "my_custom_app": {
    "url": "https://github.com/yourorg/my_custom_app.git",
    "branch": "main"
  },
  "another_app": {
    "url": "https://github.com/yourorg/another_app.git",
    "branch": "version-15"
  }
}
```

2. Create the Custom Containerfile

```
# ?? Stage 1: Builder ?????????????????????????????????????????????????????????
FROM ghcr.io/vyogotech/frappe_base:latest AS builder

USER root
RUN apt-get update && apt-get install --no-install-recommends -y \
  git build-essential python3-dev libffi-dev libmariadb-dev jq \
  && npm install -g yarn \
  && rm -rf /var/lib/apt/lists/*

USER 1000
WORKDIR /home/frappe/frappe-bench
SHELL ["/bin/bash", "-o", "pipefail", "-c"]

ARG APPS_JSON_BASE64

# Create dummy config for build orchestration
RUN mkdir -p sites && \
  echo '{"socketio_port": 9000}' > sites/common_site_config.json
```

3. Build and Push Custom Image

6. Deployment Examples

With the infrastructure ready and images built, you can now deploy your Bench and Site.

6.1 Create the FrappeBench

A Bench represents your application environment. Create `my-bench.yaml`.

www.vyogo.tech

```
kind: Secret
metadata:
  name: redis-external-creds
  namespace: frappe
type: Opaque
stringData:
  password: <your-redis-password>
```

```
apiVersion: vyogo.tech/v1alpha1
kind: FrappeBench
metadata:
  name: prod-bench
  namespace: frappe
spec:
  frappeVersion: "15.0.0"
  imageConfig:
    repository: your-registry.io/frappe/custom-apps
    tag: v15-1.0.0
    pullPolicy: Always
    pullSecrets:
      - name: registry-credentials
  # External Redis configuration
  redisConfig:
    type: redis
    external: true
    host: redis-external.redis.svc.cluster.local # update with your redis host
    port: 6379
    connectionSecretRef:
      name: redis-external-creds
  componentAutoscaling:
    unicorn:
      enabled: true
      minReplicas: 2
      maxReplicas: 5
      provider: hpa
      hpa:
        metric: cpu
        targetUtilization: 75
        scaleUpStabilization: 0
        scaleDownStabilization: 60
    nginx:
      enabled: true
      minReplicas: 2
      maxReplicas: 4
      provider: hpa
      hpa:
        metric: cpu
        targetUtilization: 80
        scaleUpStabilization: 0
        scaleDownStabilization: 60
  workerDefault:
    enabled: true
    minReplicas: 1
    maxReplicas: 5
```

```
provider: hpa
hpa:
  metric: cpu
  targetUtilization: 85
workerShort:
  enabled: false # Static replicas
  staticReplicas: 1
workerLong:
  enabled: false
  staticReplicas: 1
socketio:
  enabled: false
  staticReplicas: 1
storageSize: 5Gi
```

6.2 Create the FrappeSite

A Site is your actual application instance. Create `my-site.yaml`:

```
apiVersion: v1
kind: Secret
metadata:
  name: external-mariadb-creds
  namespace: frappe
type: Opaque
stringData:
  database: <your-database-name>
  password: <your-database-password>
  username: <your-database-username>
```

```
apiVersion: v1
kind: Secret
metadata:
  name: prod-site-admin
  namespace: frappe
type: Opaque
stringData:
  password: "AdminPassword123"
```

```
apiVersion: vyogo.tech/v1alpha1
kind: FrappeSite
metadata:
  name: prod-site
  namespace: frappe
spec:
  benchRef: prod-bench
  siteName: prod-site.apps.cluster.example.com # Must be a valid domain
  adminPasswordSecretRef:
    name: prod-site-admin
    key: password
  dbConfig:
    provider: external
    host: <Your-MariaDB-Host> # your mariaDB host
    port: "3306"
```

```
connectionSecretRef:
  name: external-mariadb-creds
routeConfig:
  enabled: true
  termination: edge
```

Apply all manifests:

```
oc apply -f my-bench.yaml
oc apply -f my-site.yaml
```

7. Verification & Troubleshooting

Check Pods and Jobs

```
oc get pods -n frappe
oc get jobs -n frappe
```

Access the Site

Once the site phase is `Ready`, retrieve the URL from the OpenShift Route:

```
oc get route prod-site -n frappe -o jsonpath='{.spec.host}'
```

For more details, see:

- [OpenShift Technical Guide](#)
- [MariaDB Integration Guide](#)